

LC 377: Combination Sum IV

Target Companies: OYO, Visa

Problem Overview

Find the number of possible combinations that add up to a specific positive integer *target*. Note that numbers are always positive.

Example:

nums = [1, 2, 3], *target* = 4

Output: 7

Explanation: The possible combination ways are obtained by taking a number an unlimited number of times. Different sequences are counted as different combinations. For example, (1, 1, 2), (1, 2, 1), and (2, 1, 1) are counted as 3 separate ways.

Dynamic Programming Approach

DP State: We cannot have an index in our DP state because once moving past it, we can never go back to it. For instance, the sequence (1, 2, 1) would never be possible if we restricted reusing previous elements via indices. Hence, we just use `solve(target)`.

Base Cases:

- If ***target == 0*** → return 1
- If ***target < 0*** → return 0

Recurrence Relation:

```
ans = 0
for (each num) {
    ans += solve(target - num)
}
```

Complexity:

Time Complexity: ***$O(\text{target} \times n)$***

C++ Implementation

```
class Solution {
public:
    vector<int> dp;
    int n;

    int solve(vector<int>& nums, int target){
        if(target == 0){
            return 1;
        }

        if(target < 0){
            return 0;
        }

        if(dp[target] != -1){
            return dp[target];
        }

        int ans = 0;
        for(int num : nums){
            ans += solve(nums, target - num);
        }

        dp[target] = ans;
        return dp[target];
    }

    int combinationSum4(vector<int>& nums, int target) {
        n = nums.size();
        dp.assign(target + 1, -1);
        return solve(nums, target);
    }
};
```